

Data Structures

Lecture 20: Graph

Soobin Um

May 27, 2026

Basics of Graph

Examples



Image credit: [Medium](#)



Image credit: [Medium](#)

Viral Marketing

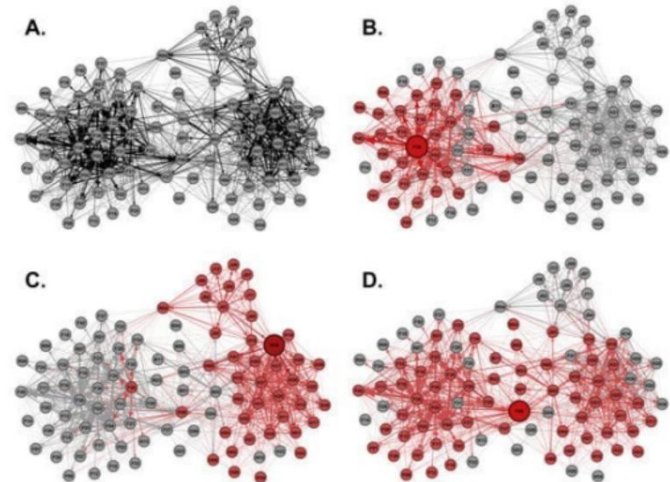
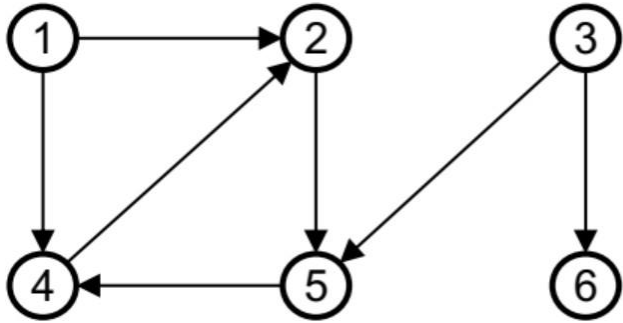


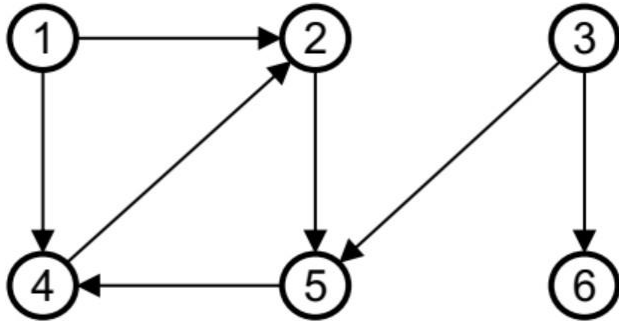
Image credit: [MDPI](#)

Infection Control

Definition of Graph



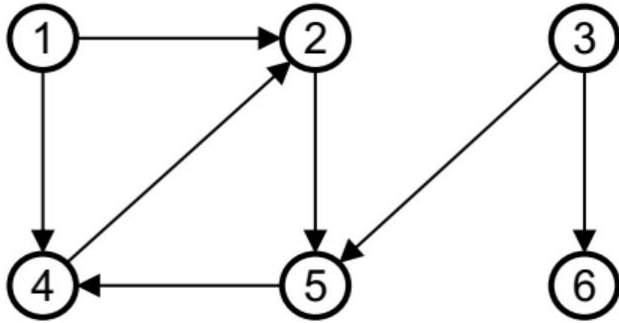
Definition of Graph



$$V = \{1,2,3,4,5,6\}$$

$$E = \{(1,2), (1,4), (2,5), (3,5), (3,6), (4,2), (5,4)\}$$

Definition of Graph

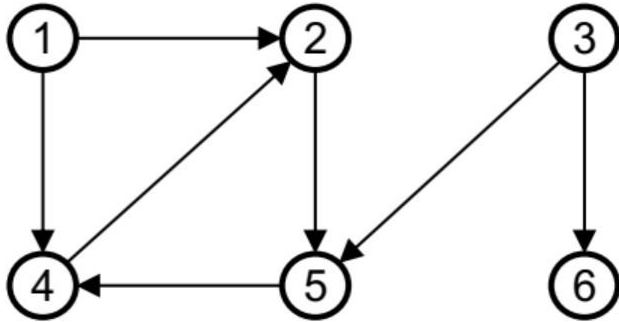


$$V = \{1,2,3,4,5,6\}$$

$$E = \{(1,2), (1,4), (2,5), (3,5), (3,6), (4,2), (5,4)\}$$

- 그래프 $G=(V, E)$ 는 다음의 두 가지 집합으로 이루어진 자료 구조
 - 정점 집합 V (Vertices)
 - 간선 집합 E (Edges)

Definition of Graph

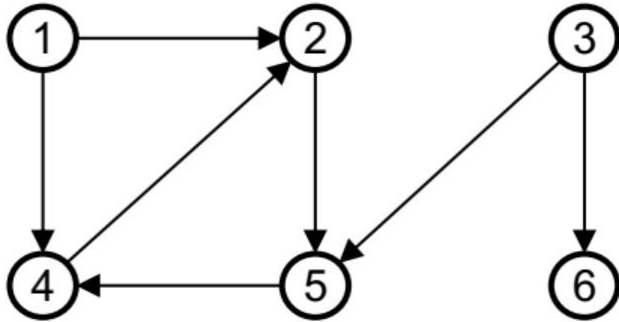


$$V = \{1,2,3,4,5,6\}$$

$$E = \{(1,2), (1,4), (2,5), (3,5), (3,6), (4,2), (5,4)\}$$

- 그래프 $G=(V, E)$ 는 다음의 두 가지 집합으로 이루어진 자료 구조
 - 정점 집합 V (Vertices)
 - 간선 집합 E (Edges)
- 간선 집합 E 의 각 원소는 정점 집합 V 에 속한 두 정점을 연결하는 관계를 의미
 - 소셜 네트워크 그래프 (사람들 간의 친구 관계)
 - 전화 통화 기록 그래프
 - 컴퓨터 네트워크 그래프 (서버·라우터 간 연결)

Definition of Graph



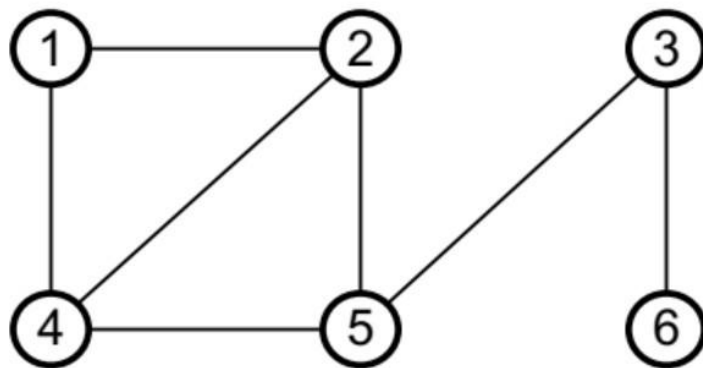
$$V = \{1,2,3,4,5,6\}$$

$$E = \{(1,2), (1,4), (2,5), (3,5), (3,6), (4,2), (5,4)\}$$

- 그래프 $G=(V, E)$ 는 다음의 두 가지 집합으로 이루어진 자료 구조
 - 정점 집합 V (Vertices)
 - 간선 집합 E (Edges)
- 간선 집합 E 의 각 원소는 정점 집합 V 에 속한 두 정점을 연결하는 관계를 의미
 - 소셜 네트워크 그래프 (사람들 간의 친구 관계)
 - 전화 통화 기록 그래프
 - 컴퓨터 네트워크 그래프 (서버·라우터 간 연결)
- 정점의 개수는 $|V|$, 간선의 개수는 $|E|$ 로 표기

Types: Undirected Graph (무향 그래프)

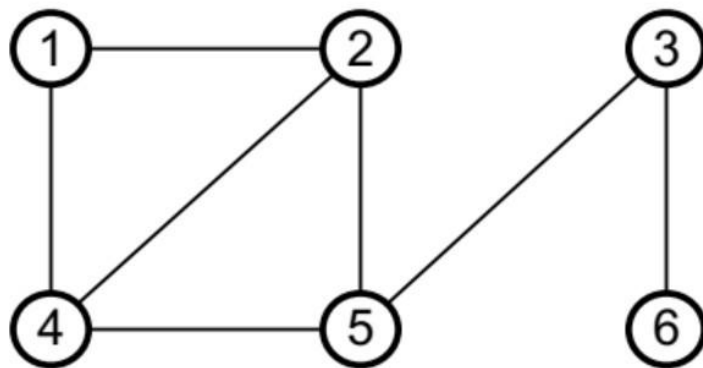
$$(4,2) = (2,4)$$



- 간선에 방향이 없는 그래프

Types: Undirected Graph (무향 그래프)

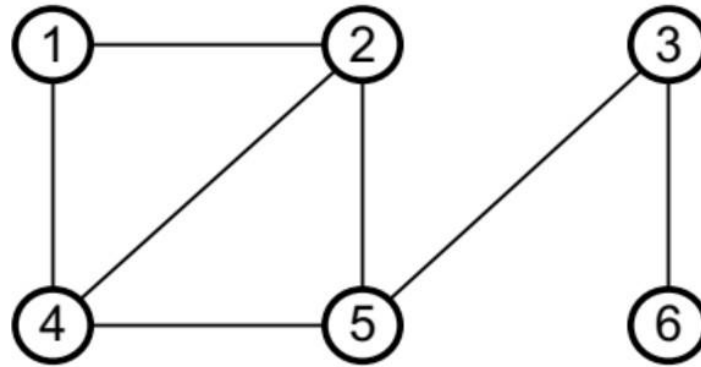
$$(4,2) = (2,4)$$



- 간선에 방향이 없는 그래프
- 정점 u 와 v 가 연결되어 있다면, $u \rightarrow v$ 및 $v \rightarrow u$ 모두 가능
 - Ex) 친구 관계, 협업 관계 등 상호 연결이 중요한 관계

Types: Undirected Graph (무향 그래프)

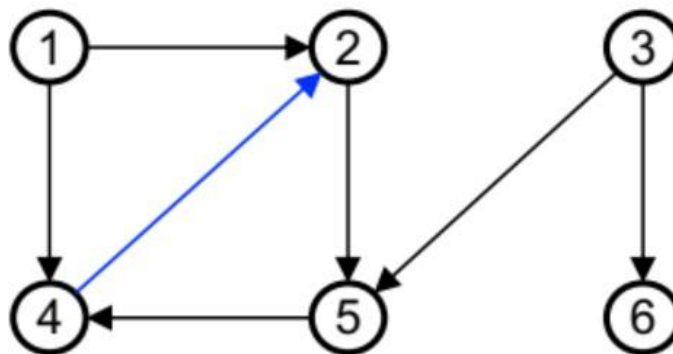
$$(4,2) = (2,4)$$



- 간선에 방향이 없는 그래프
- 정점 u 와 v 가 연결되어 있다면, $u \rightarrow v$ 및 $v \rightarrow u$ 모두 가능
 - Ex) 친구 관계, 협업 관계 등 상호 연결이 중요한 관계
- 정점의 차수(degree)는 해당 정점과 연결된 간선의 수와 같음

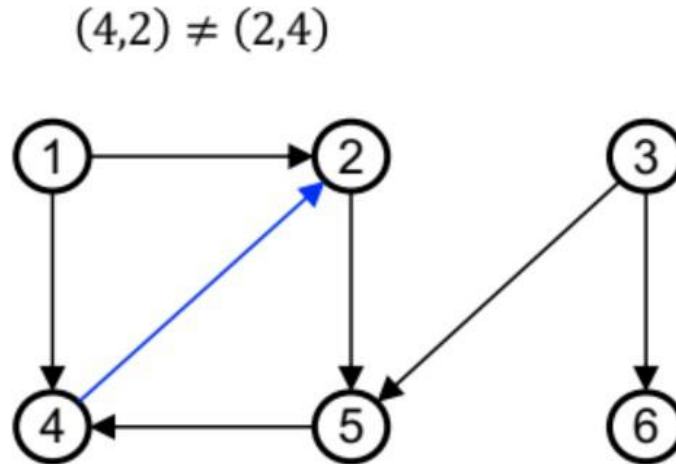
Types: Directed Graph (유향 그래프)

$(4,2) \neq (2,4)$



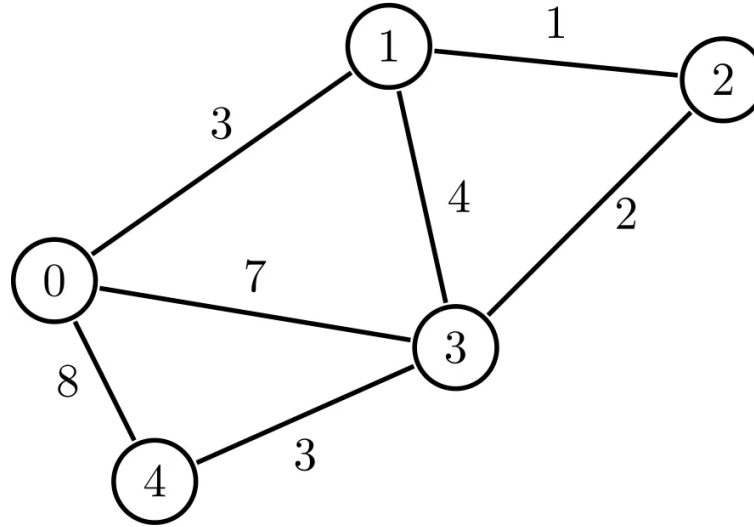
- 간선에 방향이 있는 그래프
- 간선 $u \rightarrow v$ 와 $v \rightarrow u$ 는 서로 다른 간선
 - Ex) 팔로우 관계, 웹 페이지 링크 구조, 전화 발신 기록 등

Types: Directed Graph (유향 그래프)



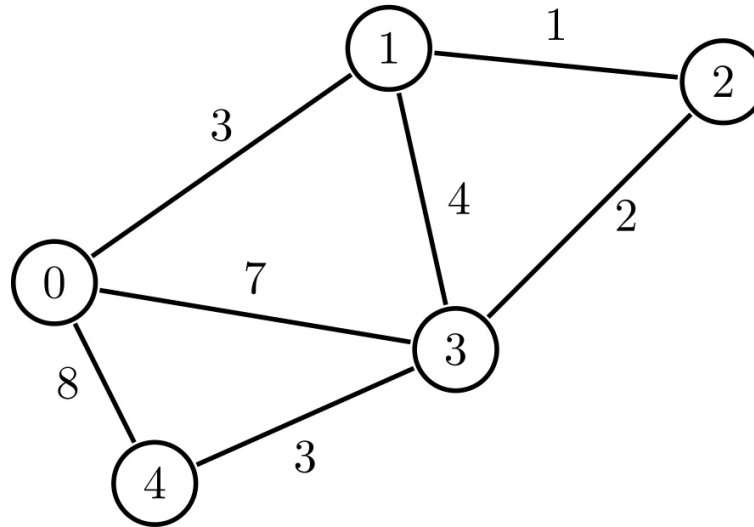
- 간선에 방향이 있는 그래프
- 간선 $u \rightarrow v$ 와 $v \rightarrow u$ 는 서로 다른 간선
 - Ex) 팔로우 관계, 웹 페이지 링크 구조, 전화 발신 기록 등
- 정점의 차수가 두 종류로 구분됨
 - 진입 차수 (in-degree): 해당 정점으로 **들어오는** 간선의 수
 - 진출 차수 (out-degree): 해당 정점에서 **나가는** 간선의 수

Types: Weighted Graph (가중치 그래프)



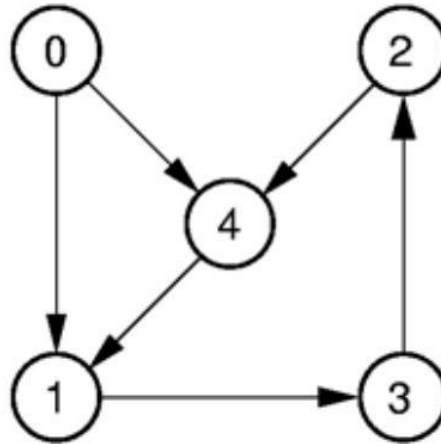
- 간선마다 비용(가중치, weight)이 부여된 그래프

Types: Weighted Graph (가중치 그래프)



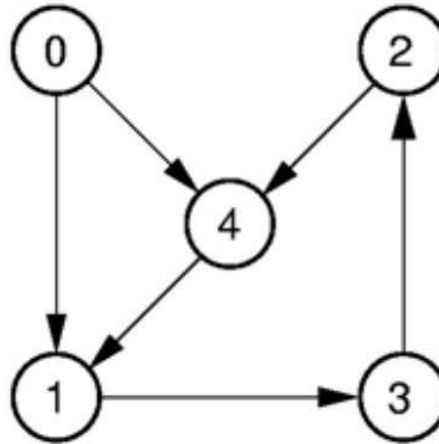
- 간선마다 비용(가중치, weight)이 부여된 그래프
- 거리, 비용 등의 값을 표현할 때 사용
 - Ex) 지도 (거리), 네트워크 비용, 교통 흐름 등

Path (경로)



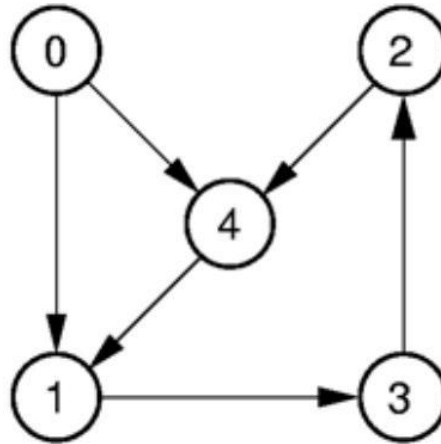
- 정점들의 나열 $v_1, v_2, \dots, v_n$ 에 대하여, 정점 v_i 와 v_{i+1} 사이에 간선이 존재하는 경우 이러한 정점들의 나열을 길이가 $n - 1$ 인 경로로 간주 (for $1 \leq i < n$)
 - 0, 4, 1, 3, 2, 4: 경로

Path (경로)



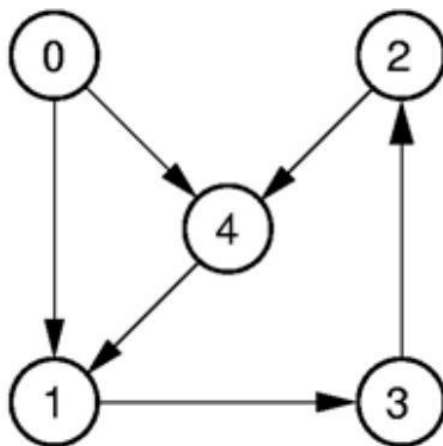
- 정점들의 나열 $v_1, v_2, \dots, v_n$ 에 대하여, 정점 v_i 와 v_{i+1} 사이에 간선이 존재하는 경우 이러한 정점들의 나열을 길이가 $n - 1$ 인 경로로 간주 (for $1 \leq i < n$)
 - 0, 4, 1, 3, 2, 4: 경로
- 경로 상의 모든 정점이 서로 중복 없이 다른 경우 → 단순 경로 (simple path)
 - 0, 4, 1, 3, 2: 단순 경로

Cycle (사이클, 순환 경로)



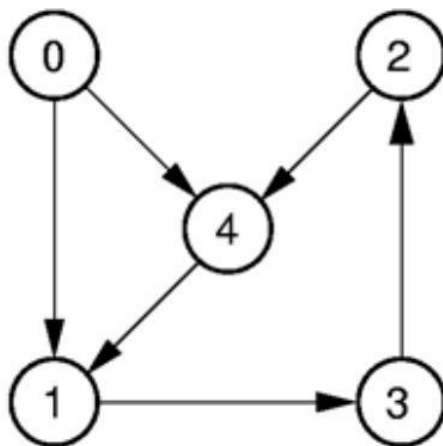
- 길이가 3 이상인 경로 중, 시작 정점 v_i 에서 출발하여 여러 정점을 거친 후, 다시 동일 정점 v_i 로 돌아오는 경로
 - 1, 3, 2, 4, 1, 3, 2, 4, 1: 사이클

Cycle (사이클, 순환 경로)



- 길이가 3 이상인 경로 중, 시작 정점 v_i 에서 출발하여 여러 정점을 거친 후, 다시 **동일 정점 v_i 로 돌아오는 경로**
 - 1, 3, 2, 4, 1, 3, 2, 4, 1: 사이클
- 경로 상에서 첫 번째와 마지막 정점을 제외하고는 **모든 정점이 서로 다른 경우**
→ **단순 사이클 (simple cycle)**
 - 1, 3, 2, 4, 1: 단순 사이클

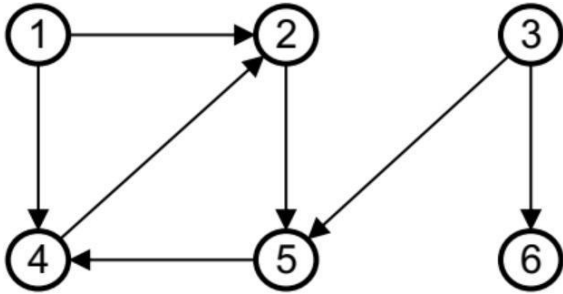
Cycle (사이클, 순환 경로)



- 길이가 3 이상인 경로 중, 시작 정점 v_i 에서 출발하여 여러 정점을 거친 후, 다시 **동일 정점 v_i 로 돌아오는 경로**
 - 1, 3, 2, 4, 1, 3, 2, 4, 1: 사이클
- 경로 상에서 첫 번째와 마지막 정점을 제외하고는 **모든 정점이 서로 다른 경우**
→ **단순 사이클 (simple cycle)**
 - 1, 3, 2, 4, 1: 단순 사이클

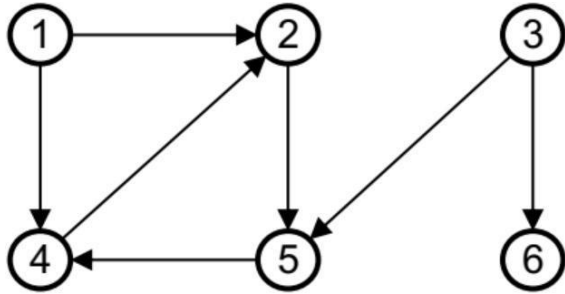
Note: 경로 및 사이클은 무향/유향 그래프 모두에서 정의될 수 있음

Acyclic Graph (비순환 그래프)

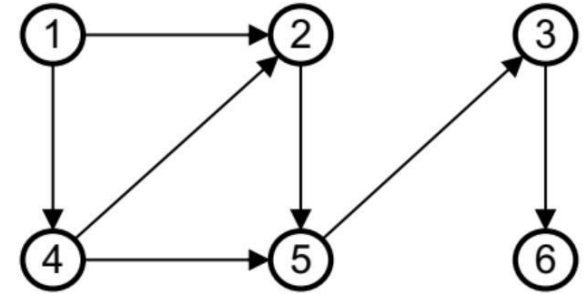


Directed graph

Acyclic Graph (비순환 그래프)



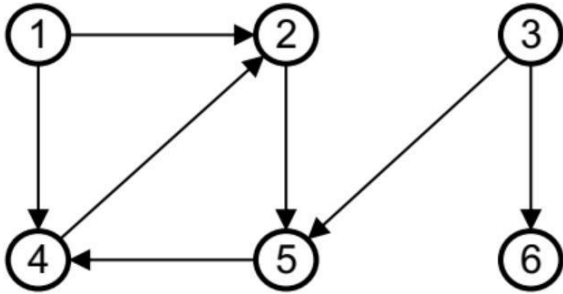
Directed graph



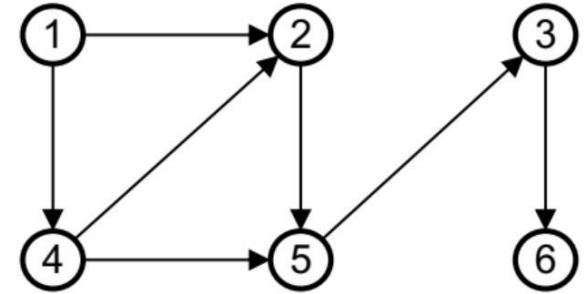
Directed acyclic graph (DAG)

- 사이클이 없는 그래프

Acyclic Graph (비순환 그래프)



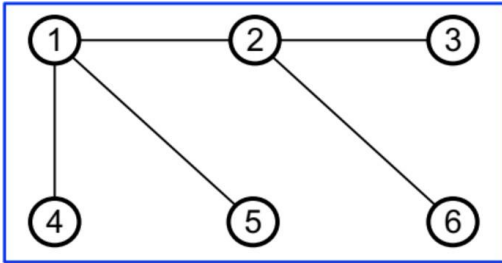
Directed graph



Directed acyclic graph (DAG)

- 사이클이 없는 그래프
- 유형 비순환 그래프 (DAG, Directed Acyclic Graph)
 - 간선에 방향이 있으며 사이클을 포함하지 않는 그래프
 - 작업 스케줄링, 의존성 관리, 정렬 순서를 찾는 위상 정렬 등에 활용

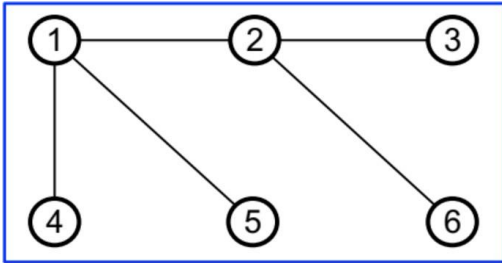
Graph vs. Tree



Graph: O

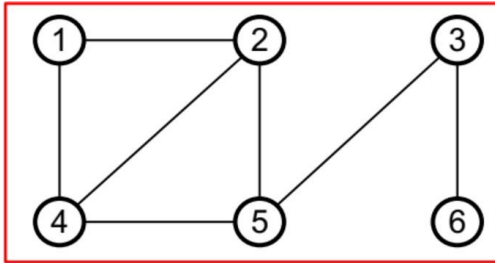
Tree: O

Graph vs. Tree



Graph: O

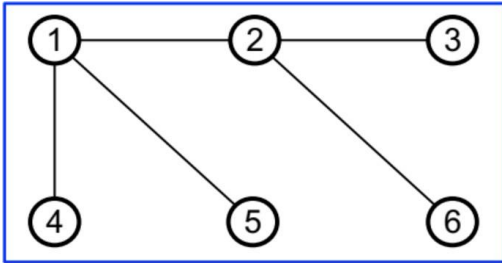
Tree: O



Graph: O

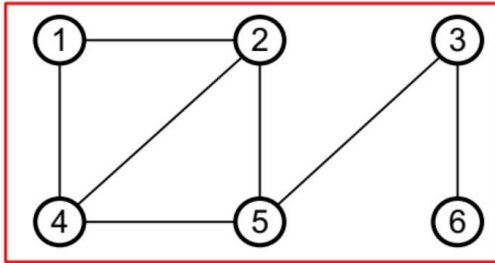
Tree: X (cyclic)

Graph vs. Tree



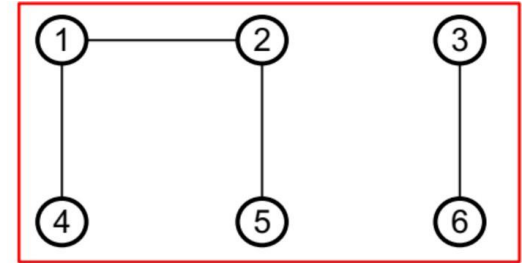
Graph: O

Tree: O



Graph: O

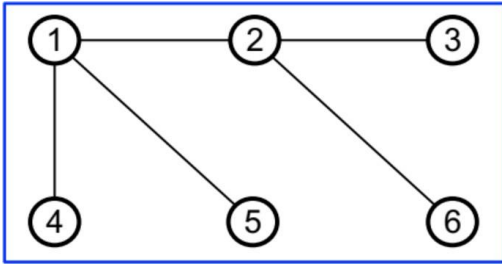
Tree: X (**cyclic**)



Graph: O

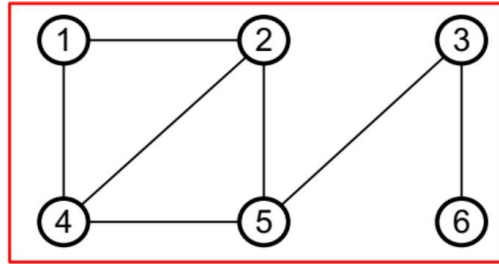
Tree: X (**not connected**)

Graph vs. Tree



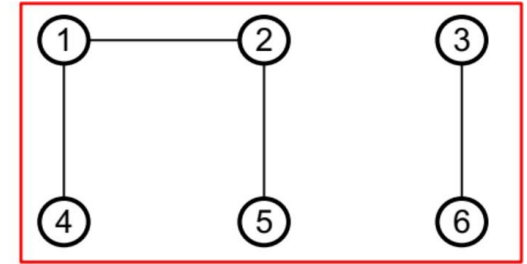
Graph: O

Tree: O



Graph: O

Tree: X (**cyclic**)

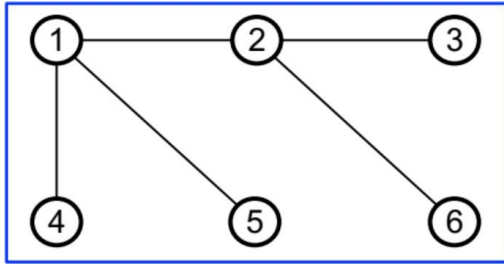


Graph: O

Tree: X (**not connected**)

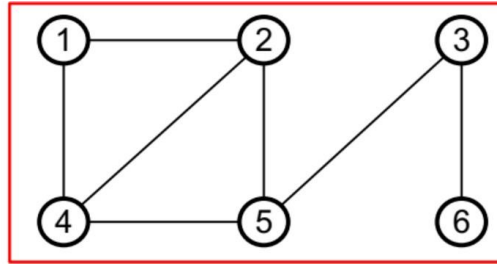
- “모든 트리는 그래프이지만, 모든 그래프가 트리는 아님”
 - 트리는 그래프의 한 종류이지만, 그래프가 트리가 되기 위해선 특정 조건을 만족해야 함

Graph vs. Tree



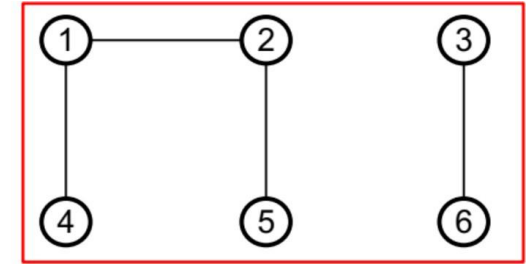
Graph: O

Tree: O



Graph: O

Tree: X (cyclic)

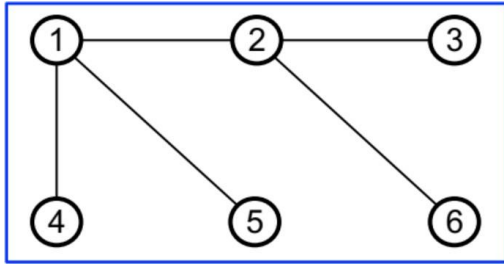


Graph: O

Tree: X (not connected)

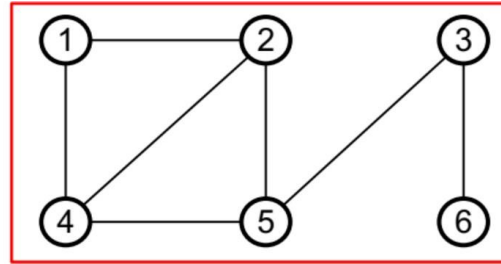
- “모든 트리는 그래프이지만, 모든 그래프가 트리는 아님”
 - 트리는 그래프의 한 종류이지만, 그래프가 트리가 되기 위해선 특정 조건을 만족해야 함
- 트리가 되기 위한 조건
 - 사이클이 없어야 함 (Acyclic)
 - 모든 정점이 서로 연결되어 있어야 함 (Connected)
 - 그래프의 모든 정점 간에 도달 가능한 경로가 존재해야 함

Graph vs. Tree



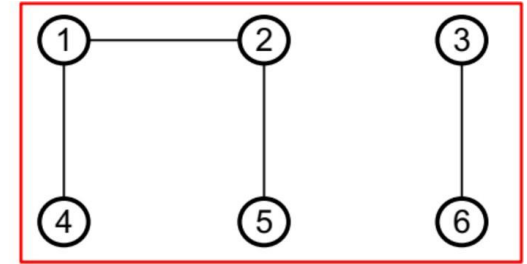
Graph: O

Tree: O



Graph: O

Tree: X (cyclic)



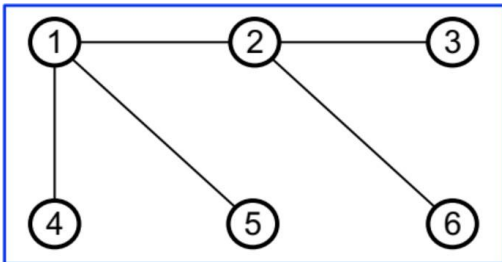
Graph: O

Tree: X (not connected)

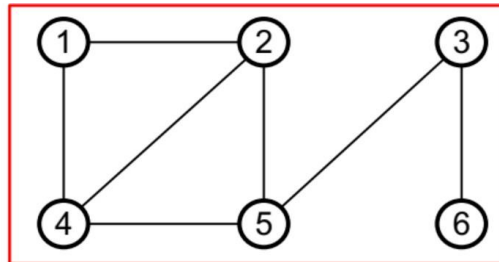
- “모든 트리는 그래프이지만, 모든 그래프가 트리는 아님”
 - 트리는 그래프의 한 종류이지만, 그래프가 트리가 되기 위해선 특정 조건을 만족해야 함
- 트리가 되기 위한 조건
 - 사이클이 없어야 함 (Acyclic)
 - 모든 정점이 서로 연결되어 있어야 함 (Connected)
 - 그래프의 모든 정점 간에 도달 가능한 경로가 존재해야 함
- (+) 트리의 중요한 성질
 - 어떤 간선 하나라도 제거하면 연결성이 사라짐 (즉, 연결 그래프가 아니게 됨)
 - 트리는 최소 연결 구조의 그래프
 - 하나의 간선 제거 시, 두 컴포넌트로 분리
 - 임의의 두 정점 사이에는 하나의 단순 경로만 존재
 - 두 정점을 연결하는 단순 경로(simple path)는 항상 정확히 1개만 존재

Summary: Graph vs. Tree

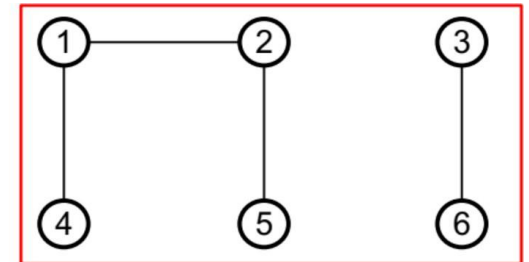
	그래프	트리
사이클 존재 여부	있을 수도 있음	없음 (acyclic)
연결성	연결될 수도 있고, 안 될 수도 있음	항상 연결
두 정점 간 단순 경로 수	여러 개일 수 있음	1개
간선 수	자유로움	정점 수가 n 이면 간선 수는 $n - 1$



Graph: O
Tree: O

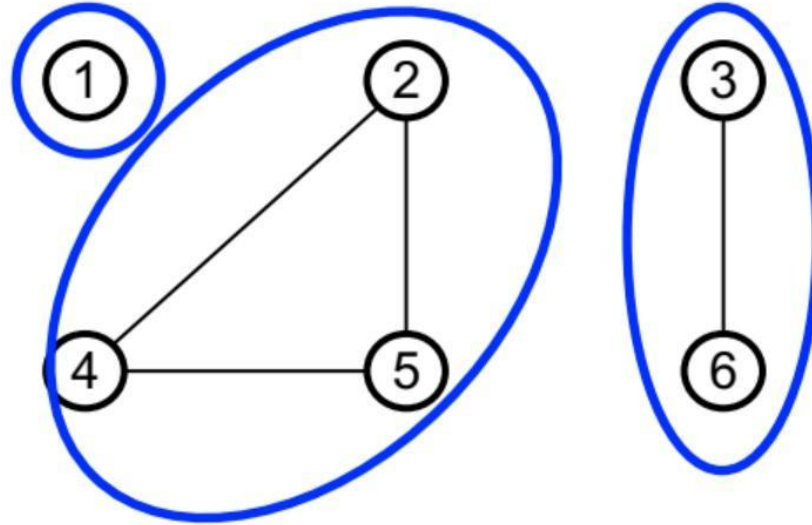


Graph: O
Tree: X (**cyclic**)

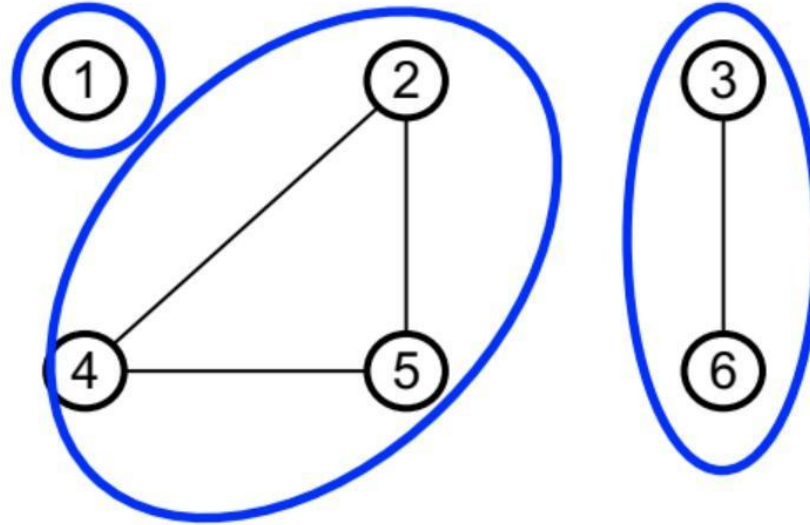


Graph: O
Tree: X (**not connected**)

Connected Components (연결 요소)



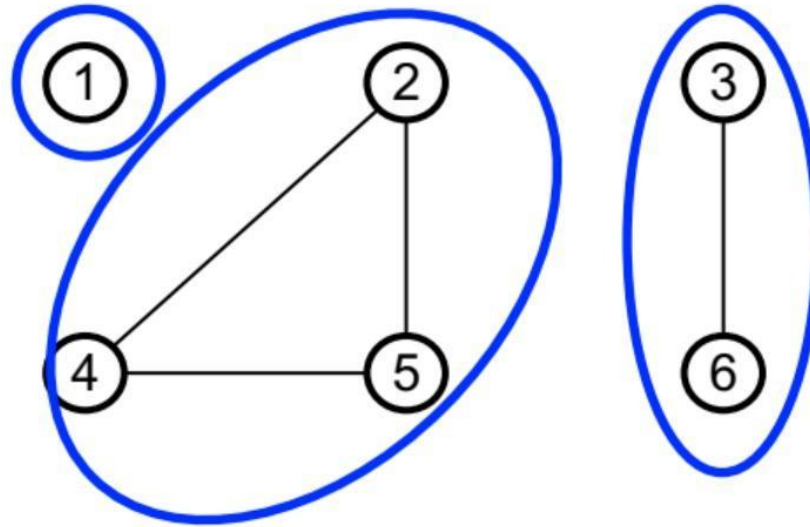
Connected Components (연결 요소)



연결요소: 3개

- 무향 그래프에서 모든 정점 쌍이 어떤 경로로든 서로 연결되어 있는 부분 그래프
 - 서로 다른 연결 요소 사이에는 경로가 없음

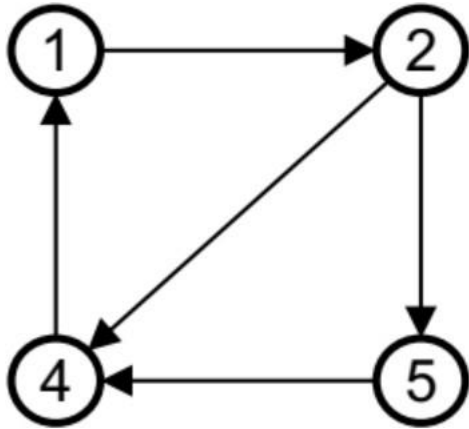
Connected Components (연결 요소)



연결요소: 3개

- 무향 그래프에서 모든 정점 쌍이 어떤 경로로든 서로 연결되어 있는 부분 그래프
 - 서로 다른 연결 요소 사이에는 경로가 없음
- 더 이상 확장할 수 없는 최대 연결 정점 집합

Strongly Connected Components (SCC; 강연결요소)

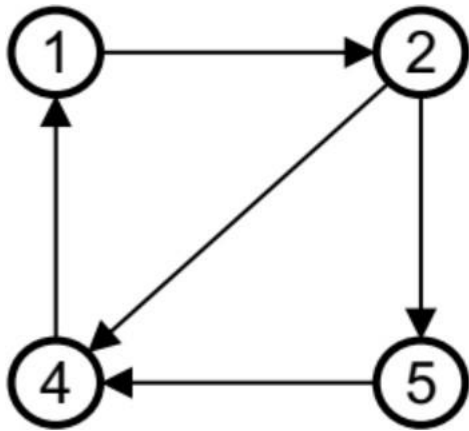


SCC 1: {1, 2, 4, 5}

SCC 2: {3}

SCC 3: {6}

Strongly Connected Components (SCC; 강연결요소)



SCC 1: {1, 2, 4, 5}

SCC 2: {3}

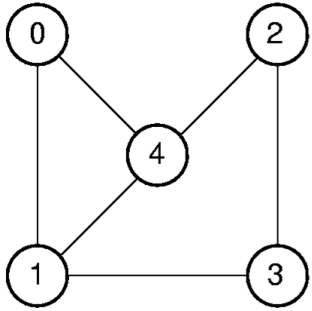
SCC 3: {6}

- 유향 그래프에서 임의의 두 정점 u, v 에 대하여 $u \rightarrow v, v \rightarrow u$ 의 양방향으로 모두 도달 가능한 경우, 해당 정점들은 같은 강연결 요소에 속함
- 더 이상 확장할 수 없는 최대 강연결 정점 집합을 의미

Graph Representation

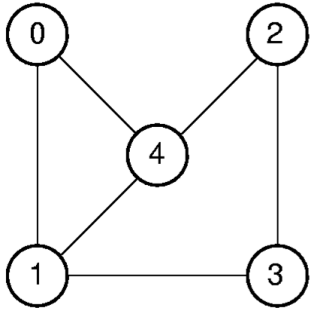
(그래프 표현 방법)

Adjacency-Matrix Representation (인접 행렬 표현법)



Undirected

Adjacency-Matrix Representation (인접 행렬 표현법)

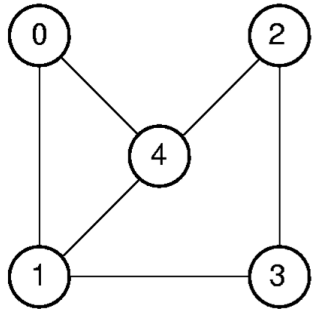


Undirected

	0	1	2	3	4
0		1			1
1	1			1	1
2				1	1
3		1	1		
4	1	1	1		

Adjacency matrix
(symmetric)

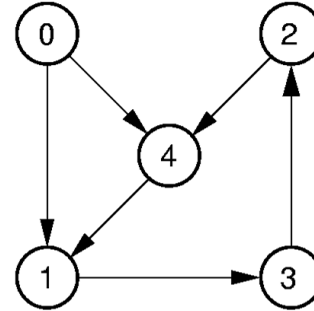
Adjacency-Matrix Representation (인접 행렬 표현법)



Undirected

	0	1	2	3	4
0		1			1
1	1			1	1
2				1	1
3		1	1		
4	1	1	1		

Adjacency matrix
(symmetric)

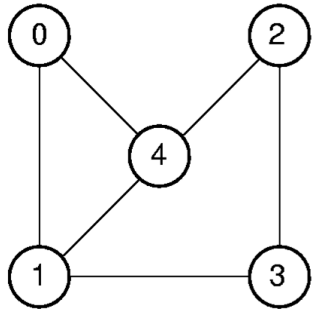


Directed

	0	1	2	3	4
0		1			1
1				1	
2					1
3		1			
4	1				

Adjacency matrix

Adjacency-Matrix Representation (인접 행렬 표현법)

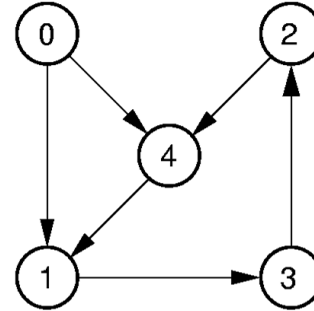


Undirected

	0	1	2	3	4
0		1			1
1	1			1	1
2					1
3		1	1		
4	1	1	1		

Adjacency matrix

(symmetric)



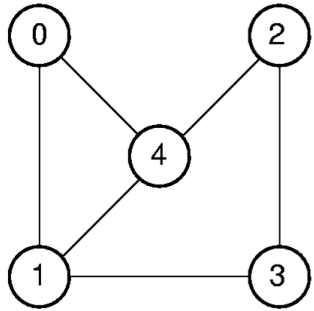
Directed

	0	1	2	3	4
0		1			1
1				1	
2					1
3			1		
4	1				

Adjacency matrix

- 정점의 개수가 $|V|$ 일 때, $|V| \times |V|$ 크기의 행렬 A 를 사용하여 그래프를 표현
 - 행렬의 원소 $A[i][j] = 1$ 일 경우, 정점 i 에서 j 로의 간선이 존재함을 의미
 - 간선이 없을 경우 0 할당

Adjacency-Matrix Representation (인접 행렬 표현법)

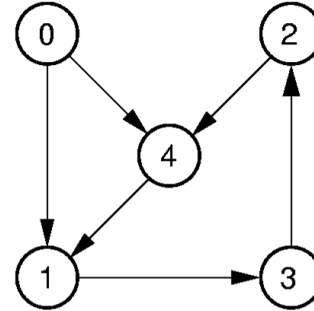


Undirected

	0	1	2	3	4
0		1			1
1	1			1	1
2					1
3		1	1		
4	1	1	1		

Adjacency matrix

(symmetric)



Directed

	0	1	2	3	4
0		1			1
1				1	
2					1
3			1		
4	1				

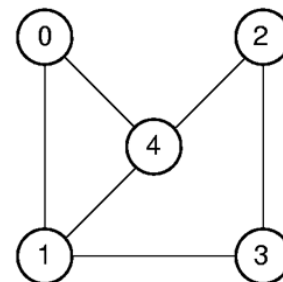
Adjacency matrix

- 정점의 개수가 $|V|$ 일 때, $|V| \times |V|$ 크기의 행렬 A 를 사용하여 그래프를 표현
 - 행렬의 원소 $A[i][j] = 1$ 일 경우, 정점 i 에서 j 로의 간선이 존재함을 의미
 - 간선이 없을 경우 0 할당
- 무향 그래프의 경우, 항상 $A[i][j] = A[j][i]$ 가 성립됨(symmetric matrix)

인접 행렬 표현법의 복잡도

- 공간 복잡도: $O(|V|^2)$

- 인접 행렬은 $|V| \times |V|$ 크기의 2차원 배열을 사용
- 간선이 없는 경우에도 해당 위치에 0을 저장
→ 희소 (sparse) 그래프에서는 비효율적



Undirected

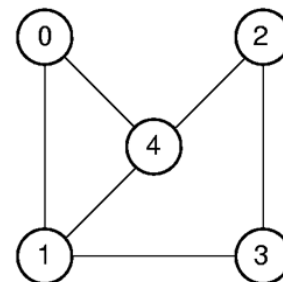
	0	1	2	3	4
0		1			1
1	1			1	1
2					1
3		1	1		
4	1	1	1		

Adjacency matrix

인접 행렬 표현법의 복잡도

- 공간 복잡도: $O(|V|^2)$

- 인접 행렬은 $|V| \times |V|$ 크기의 2차원 배열을 사용
- 간선이 없는 경우에도 해당 위치에 0을 저장
→ 희소 (sparse) 그래프에서는 비효율적



Undirected

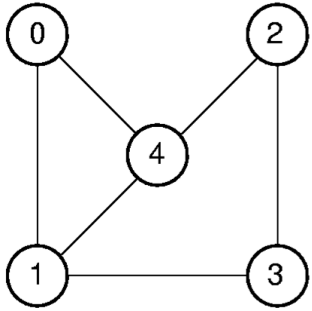
	0	1	2	3	4
0		1			1
1	1			1	1
2				1	1
3		1	1		
4	1	1	1		

Adjacency matrix

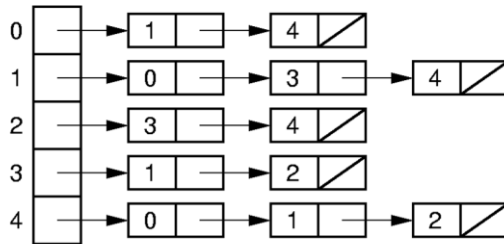
- 시간 복잡도

- 간선 존재 여부 검사: $O(1)$
 - $A[i][j] = 1$ 여부 확인 → 단순 배열 접근
- 특정 정점의 이웃 탐색: $O(|V|)$
 - 정점 i 의 이웃을 찾기 위해선 행렬의 i 번째 행 전체를 검사해야 함
- 모든 간선 나열: $O(|V|^2)$
 - 행렬 전체를 검사해야 함

Adjacency-List Representation (인접 리스트 표현법)

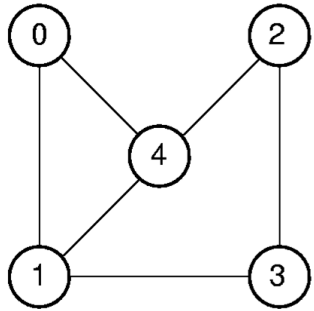


Undirected

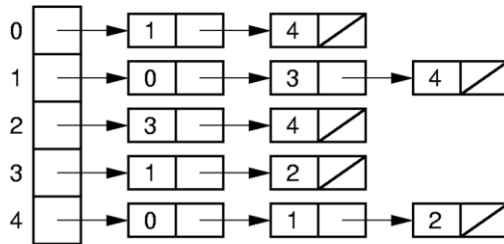


Adjacency list

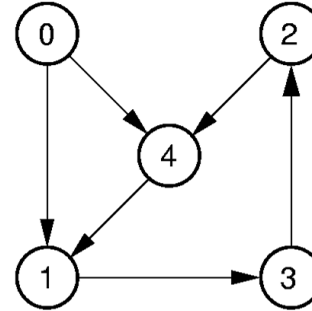
Adjacency-List Representation (인접 리스트 표현법)



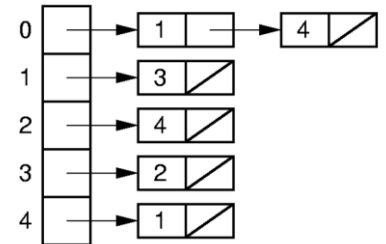
Undirected



Adjacency list

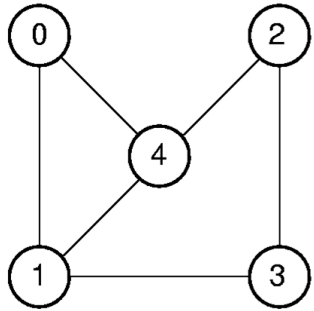


Directed

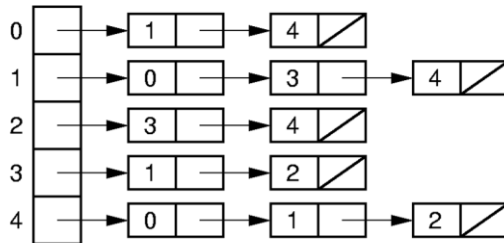


Adjacency list

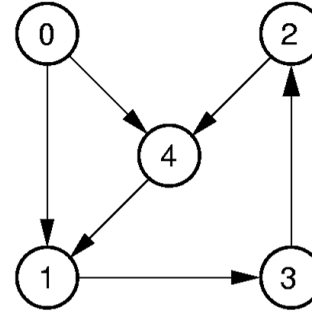
Adjacency-List Representation (인접 리스트 표현법)



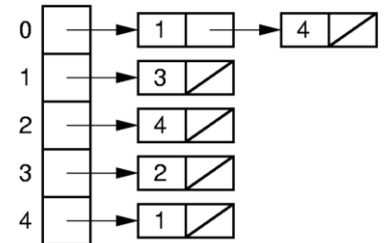
Undirected



Adjacency list



Directed



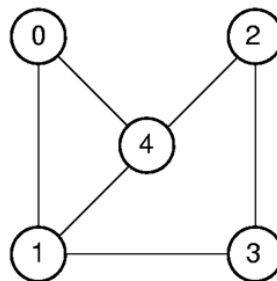
Adjacency list

- 각 정점마다 1개의 리스트를 갖도록 총 $|V|$ 개의 리스트로 이루어진 배열을 사용
- 정점 u 의 인접 리스트에는 정점 u 와 간선으로 연결된 모든 이웃 정점들이 저장됨
 - 즉, 정점별로 연결된 이웃들을 모아둔 리스트 구조

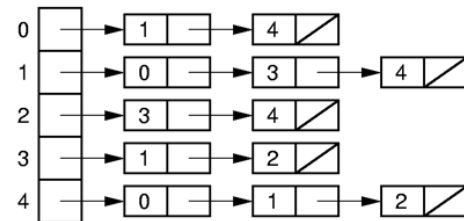
인접 리스트 표현법의 복잡도

- 공간 복잡도: $O(|V|+|E|)$

- 인접 리스트는 존재하는 간선만 저장
→ 희소 그래프에 매우 효율적



Undirected

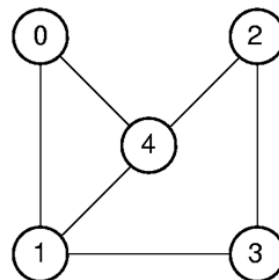


Adjacency list

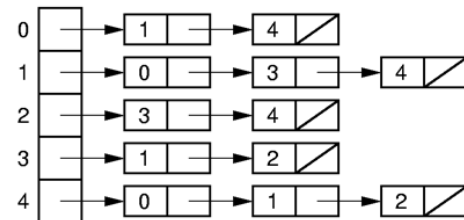
인접 리스트 표현법의 복잡도

- 공간 복잡도: $O(|V|+|E|)$

- 인접 리스트는 존재하는 간선만 저장
→ 희소 그래프에 매우 효율적



Undirected



Adjacency list

- 시간 복잡도

- 간선 존재 여부 검사: $O(\text{degree}(i))$
 - 정점 i 의 인접 리스트를 탐색해야 함
- 특정 정점의 이웃 탐색: $O(\text{degree}(i))$
 - 정점 i 의 인접 리스트를 탐색해야 함
- 모든 간선 나열: $O(|E|)$
 - 모든 리스트를 훑으며 간선을 읽으면 됨

Adjacency-Matrix vs. Adjacency-List

	시간 복잡도			공간 복잡도
	간선 존재 여부	이웃 탐색	모든 간선 탐색	
인접 행렬	$O(1)$	$O(V)$	$O(V ^2)$	$O(V ^2)$
인접 리스트	$O(\text{deg})$	$O(\text{deg})$	$O(E)$	$O(V + E)$

희소(sparse) 그래프에선 인접 리스트 표현법
밀집(dense) 그래프에선 인접 행렬 표현법이 더 유리함

Look ahead

Graph Traversal